

REPORTE DE AVANCE

Proyecto Full Stack Web & Mobile con IA

WinniKnight: MindGuardians

Aplicación RPG de Bienestar — Web + Android + IA

Integrante	Carne	Correo
Pablo Urbina	24001508	24001508@galileo.edu
Angel Camargo	24003664	angel.camargo@galileo.edu

Curso	Full Stack Web & Mobile with AI Integration
Plataformas	Web (Next.js) · Android (Kotlin / Jetpack Compose)
Backend / DB	Firebase Auth · Firebase Firestore
Inteligencia Artificial	Google Gemini 2.0 Flash API
Entrega	Fases 1 – 4 completadas
Fecha	5 de abril de 2026

Estado general del proyecto

✓ UI Web	✓ UI Android	✓ Firebase	✓ Gemini AI	✓ Documentación
----------	--------------	------------	-------------	-----------------

Contenido

1.	Descripción del Proyecto
2.	Fase 1 — Interfaz Web (Next.js)
3.	Fase 2 — Aplicación Android (Jetpack Compose)
4.	Fase 3 — Integración Firebase
5.	Fase 4 — Integración Gemini AI
6.	Documentación generada
7.	Arquitectura y estructura de datos
8.	Problemas encontrados y estrategias aplicadas
9.	Pendientes y próximos pasos

1. Descripción del Proyecto

MindGuardians (también conocido como WinniKnight) es una aplicación de bienestar gamificada que combina mecánicas de RPG con hábitos de salud reales. El jugador derrota monstruos que representan malos hábitos realizando acciones de salud en la vida real. La plataforma existe en dos interfaces sincronizadas: una aplicación web y una aplicación Android nativa.

Concepto principal

- El jugador tiene un héroe con HP, XP, nivel y oro.
- Cada hábito completado (ejercicio, hidratación, sueño, mente) se convierte en un ataque contra un monstruo.
- El Oráculo de Gemini narra las hazañas y otorga bonificadores épicos mediante IA.
- El progreso se persiste en Firestore y es visible en un ranking global.



FASE 1

Interfaz Web — Next.js + Tailwind + Shadcn/ui

2. Interfaz Web (Next.js)

Se tomó un mockup de Figma de un dashboard de salud y se convirtió en una aplicación web real. La restricción principal fue que la UI debía reflejar el mockup con precisión y quedar lista para conectar datos reales en una fase posterior.

Stack utilizado

Framework	Next.js 14 con App Router
Lenguaje	TypeScript
UI	Shadcn/ui + Tailwind CSS
Gráficas	Recharts
Íconos	Lucide React
Versiones	React 18 (compatible con lucide-react@0.383)

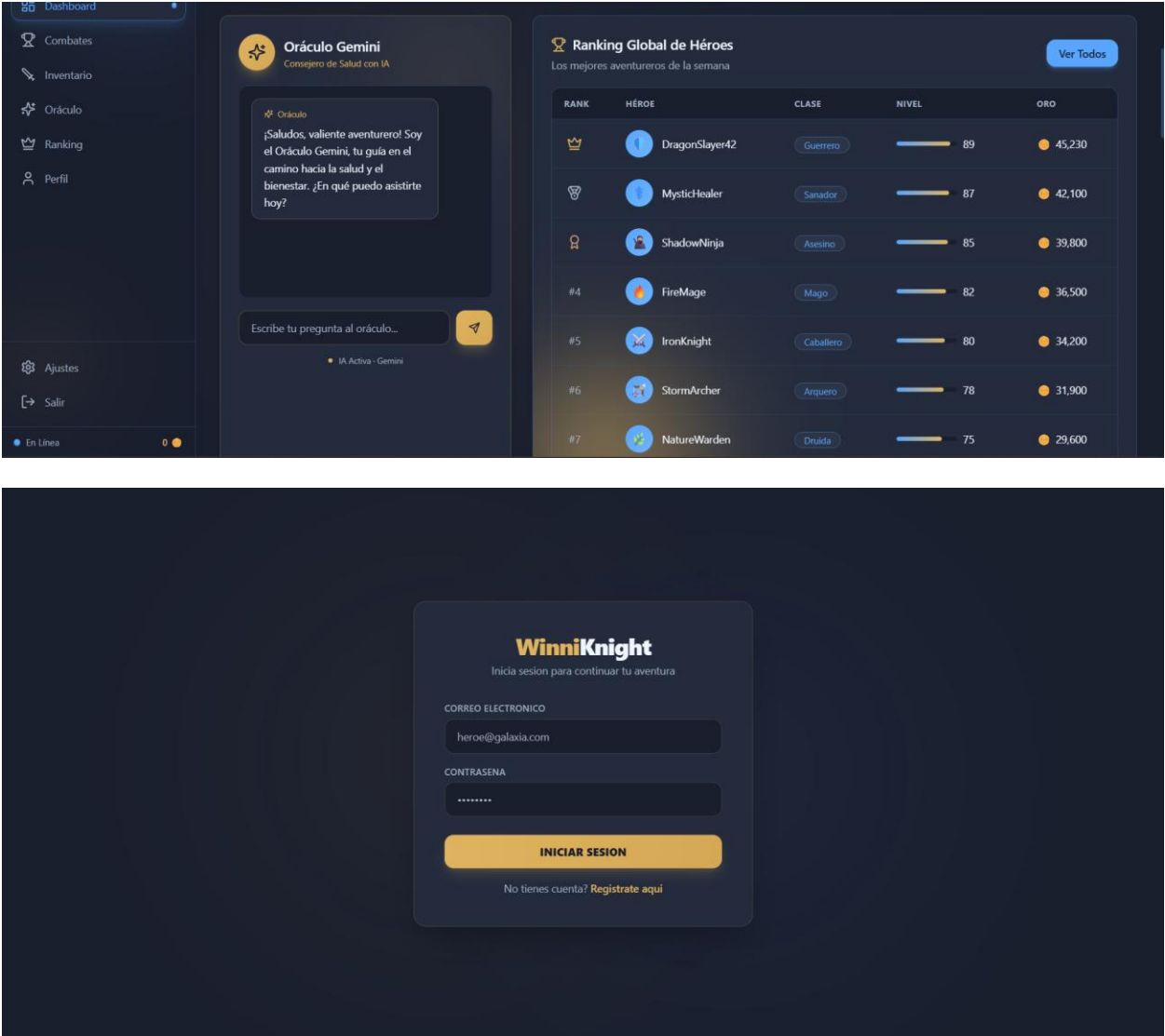
Componentes implementados

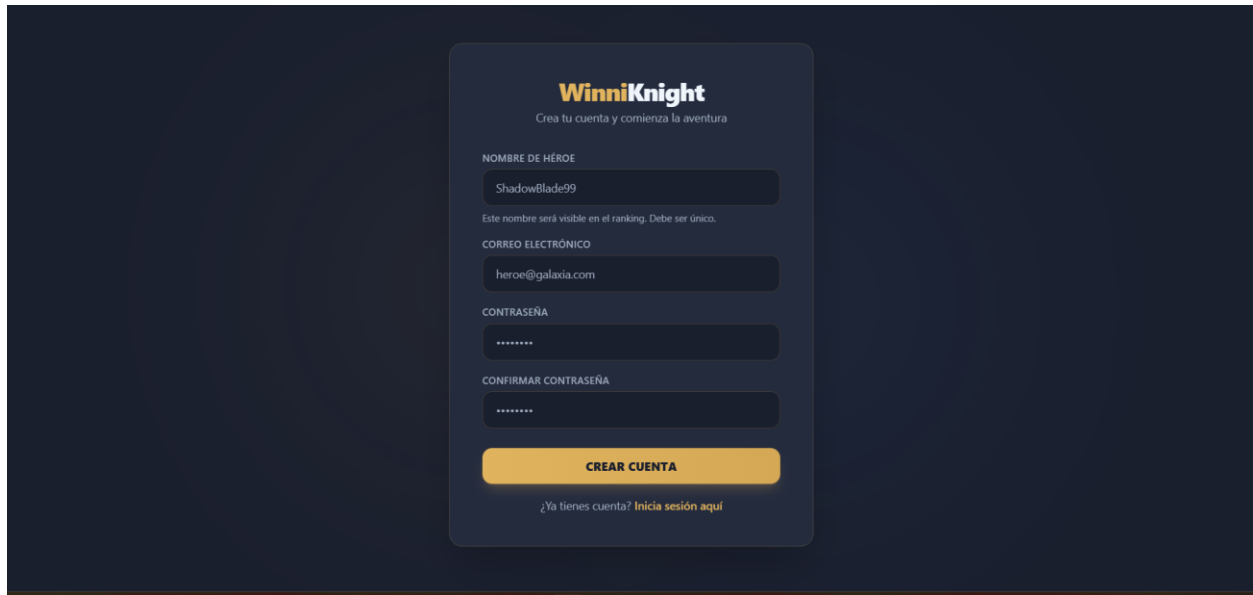
Archivo	Descripción
Sidebar.tsx	Navegación lateral con estado activo y nivel del héroe
TopBar.tsx	Barra superior con búsqueda, energía, XP y perfil
WelcomeBanner.tsx	Banner de bienvenida con stats del héroe
VitalityStats.tsx	Gráficas de barras semanales (peso y actividad)
GeminiOracle.tsx	Chat interactivo del Oráculo (conectado a Gemini)
HeroesRanking.tsx	Tabla de clasificación con medallas para top 3
EquipmentInventory.tsx	Inventario con items por rareza (common/rare/epic/legendary)
BattlesHistory.tsx	Historial de combates con filtros por tipo y fecha
GoldShop.tsx	Tienda filtrable por categoría con estado canAfford

Estructura del proyecto

src/app/	Páginas Next.js (App Router) — dashboard, login, register
src/components/	Componentes separados por sección (layout, dashboard, oracle, etc.)
src/lib/types/	Tipos TypeScript: User, Battle, Equipment, ShopItem, etc.
src/lib/data/	mock.ts — datos de prueba sustituibles por fetch() real

src/lib/firebase.ts	Inicialización de Firebase
src/lib/firestore.ts	Funciones de lectura/escritura de Firestore
src/lib/auth.ts	login(), register(), logout() con Firebase Auth



A registration form for WinniKnight on a dark blue background. The form is centered and has a dark blue card-like background. At the top, the WinniKnight logo is displayed in yellow and white, with the tagline 'Crea tu cuenta y comienza la aventura' below it. The form contains four input fields: 'NOMBRE DE HÉROE' with the value 'ShadowBlade99', 'CORREO ELECTRÓNICO' with the value 'heroe@galaxia.com', 'CONTRASEÑA', and 'CONFIRMAR CONTRASEÑA'. Each input field has a small hint or error message below it. The 'CONTRASEÑA' and 'CONFIRMAR CONTRASEÑA' fields are masked with dots. At the bottom of the form is a yellow button labeled 'CREAR CUENTA' and a link that says '¿Ya tienes cuenta? Inicia sesión aquí'.

Paleta de colores

mq-bg	#1a1f2e — Fondo oscuro principal
mq-card	#242b3d — Fondo de tarjetas
mq-blue	#58a6ff — Azul (botones, bordes activos)
mq-gold	#e0b35e — Dorado (XP, tienda, oráculo)
mq-text	#f0f6fc — Texto blanco
mq-muted	#a0aec0 — Texto gris/apagado

FASE 2

Aplicación Android — Kotlin + Jetpack Compose

3. Aplicación Android

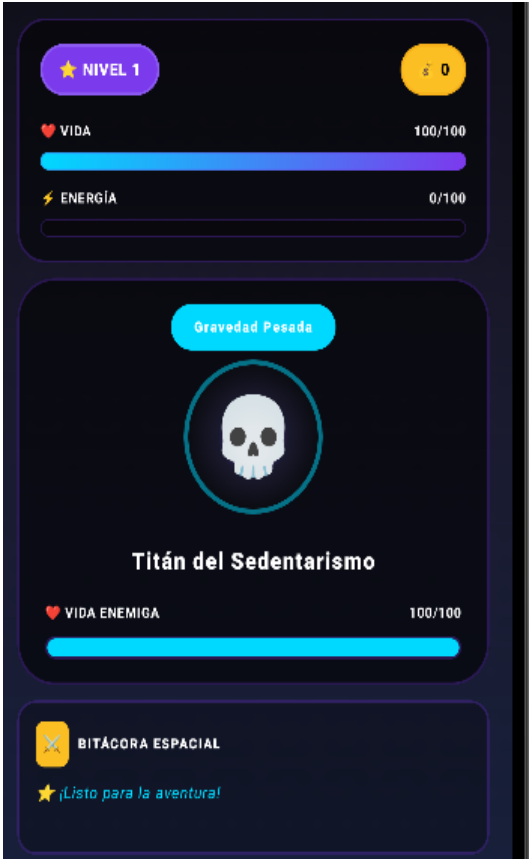
Se desarrolló la aplicación móvil fiel al mockup de Figma, adaptada a pantalla de teléfono (sin sidebar de PC). La arquitectura sigue el patrón MVVM con Jetpack Compose como sistema de UI.

Stack utilizado

Lenguaje	Kotlin
UI	Jetpack Compose + Material 3
Arquitectura	MVVM — GameViewModel centraliza todo el estado
Navegación	Navigation Compose con enum Screen
Tipografías	Orbitron (títulos) + Exo 2 (cuerpo)
Build system	Gradle con Version Catalog (libs.versions.toml)

Pantallas implementadas

Archivo	Descripción
CombatScreen	Stats del héroe (HP/Energía con barras animadas), tarjeta del monstruo flotante, 3 botones de ataque (Agua/Postura/Mente), modal de Victoria
DashboardScreen	Filtros por hábito, grid de stats, gráfica de barras semanal, historial filterable
ShopScreen	Grid 2 columnas, badge 'Equipado', precios con estado canAfford
RankingScreen	Podio visual top 3 + lista scrollable con fila del usuario en cian
OracleScreen	Chat con burbujas diferenciadas, badge de bonificador, botón de consulta conectado a Gemini
SideMenu	Menú lateral deslizante desde la derecha con perfil, navegación y stats rápidos





ORÁCULO DE GEMINI

Cuéntale tus hazañas al Narrador

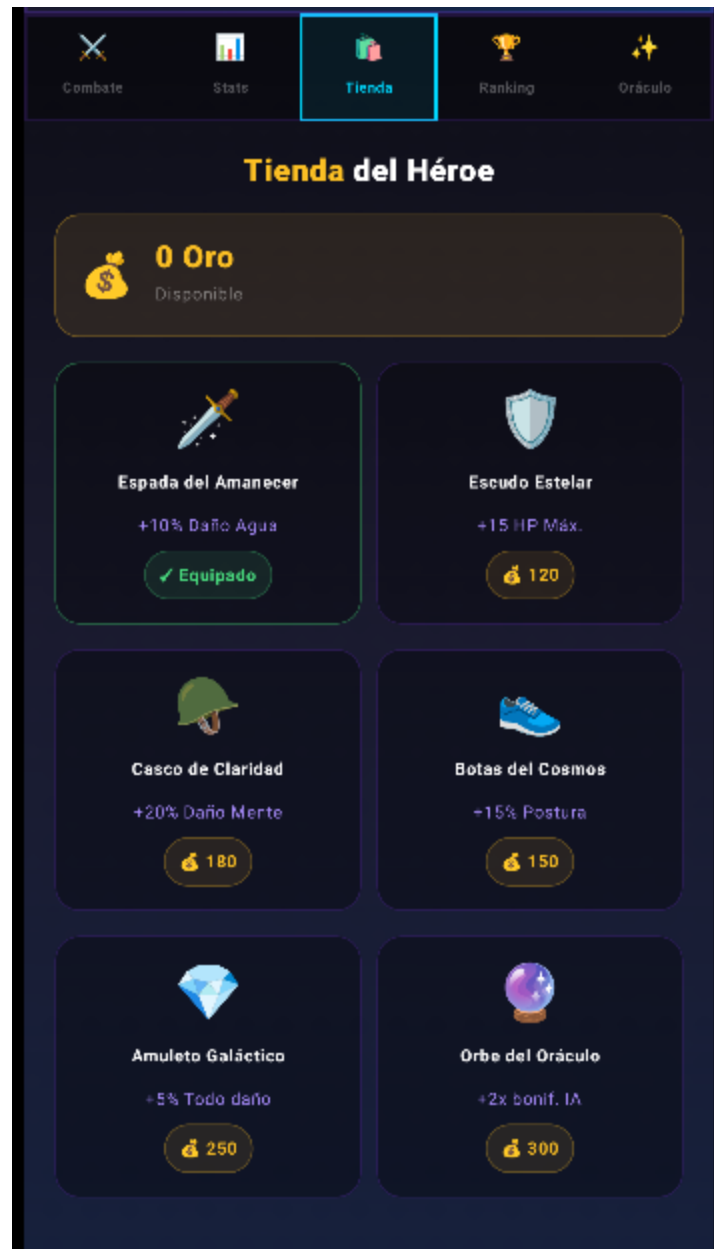
 Oráculo

¡Salve, Guerrero Estelar! El cosmos observa tu jornada. ¿Qué hazañas de salud has realizado hoy?

¿QUÉ HAZAÑA REALIZASTE HOY?

Ej: Dormí 8 horas, tomé agua, medité...

 Consultar al Oráculo



FASE 3 Integración Firebase — Auth + Firestore

4. Integración Firebase

Se integró Firebase tanto en la aplicación Android como en la web, siguiendo un proceso ordenado archivo por archivo para evitar errores de dependencias.

Archivos modificados — Android

Archivo	Descripción
<code>libs.versions.toml</code>	Versiones de firebase-bom (33.1.0), google-services (4.4.2), retrofit y GSON
<code>build.gradle.kts (root)</code>	Plugin google-services apply false
<code>app/build.gradle.kts</code>	Dependencias firebase-auth-ktx, firebase-firestore-ktx, retrofit, converter-gson
<code>AndroidManifest.xml</code>	Permiso INTERNET agregado
<code>FirebaseRepository.kt</code>	Nuevo archivo: loadUserData(), saveUserData(), saveBattle(), loadRanking()
<code>GameState.kt</code>	Integración del repositorio, loadUser() y escrituras en continueAfterVictory()
<code>.gitignore</code>	Ignorar .gradle/, build/, .idea/, google-services.json

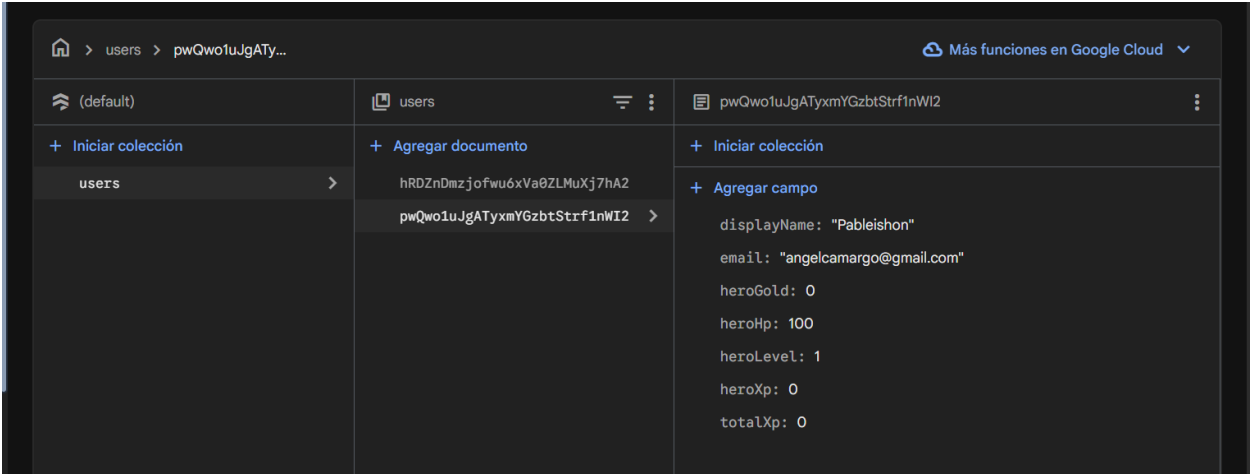
Archivos creados/modificados — Web

Archivo	Descripción
<code>src/lib/firebase.ts</code>	Inicialización de la app Firebase con variables de entorno
<code>src/lib/firestore.ts</code>	loadUserData(), saveUserData(), saveBattle(), loadRanking()
<code>src/lib/auth.ts</code>	login(), register(), logout() usando Firebase Auth
<code>.env.local</code>	Variables NEXT_PUBLIC_FIREBASE_* (no se sube a GitHub)
<code>src/app/login/page.tsx</code>	Pantalla de login con validación y mapError() en español
<code>src/app/register/page.tsx</code>	Pantalla de registro
<code>src/app/page.tsx</code>	Dashboard con onAuthStateChanged(), carga de datos reales y redirect a /login

Estructura de Firestore

Colección	users
Documento	{uid}
Campos del héroe	heroLevel, heroXp, heroGold, heroHp, totalXp, username, displayName

Subcolección	battles → { date, habitType, result, goldEarned, xpEarned }
Ranking	Consulta orderBy('totalXp').limit(10) sobre la colección users



FASE 4

Integración Gemini AI — Oracle Hello World

5. Integración Gemini AI

Se conectó la API de Gemini al Oráculo del juego en la aplicación Android. El Oráculo recibe la descripción de una hazaña del jugador y devuelve una narración épica en español junto con un bonificador de daño.

Archivos modificados — Android

Archivo	Descripción
libs.versions.toml	Retrofit 2.11.0 y converter-gson agregados
app/build.gradle.kts	implementation(libs.retrofit.core) y implementation(libs.retrofit.gson)
GeminiRepository.kt	Nuevo: llamada HTTP REST a Gemini con data classes privadas para request/response
GameState.kt	consultOracle() con estado isOracleLoading y oracleResponse
Screens.kt	OracleScreen conectada al ViewModel (campo de texto + botón funcionales)
local.properties	API_KEY=... (nunca se sube a GitHub)
app/build.gradle.kts	buildConfigField para exponer API_KEY de forma segura

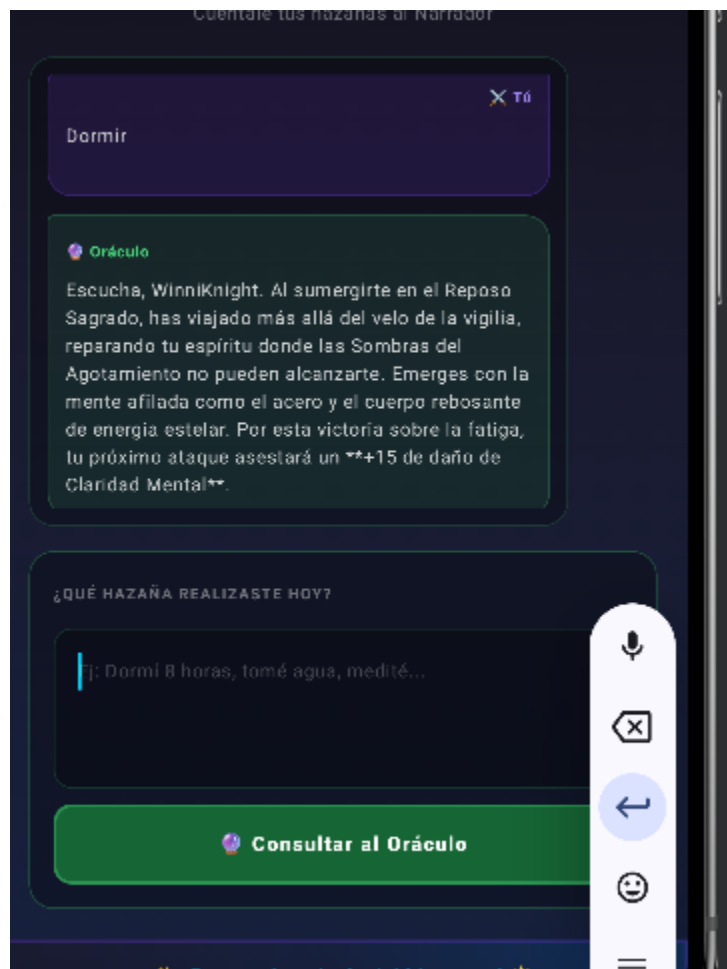
Prompt del Oráculo

El prompt enviado a Gemini le da contexto completo sobre el juego, forzando respuestas temáticas y breves:

"Eres el Oráculo de Gemini en un juego RPG de bienestar. El jugador derrota monstruos que representan malos hábitos. Responde con una narración épica en español, máximo 3 oraciones, y otorga un bonificador de daño."

Seguridad del API Key

- La clave NO está en el código fuente ni en el repositorio.
- Se almacena en local.properties (ignorado por .gitignore).
- Se expone internamente mediante BuildConfig.GEMINI_API_KEY en tiempo de compilación.
- En la web, se usa una variable de entorno NEXT_PUBLIC_ que tampoco se sube.



DOC

Documentación generada

6. Documentación Generada

Al finalizar cada plataforma se generó documentación técnica completa en dos formatos.

Android

Archivo	Descripción
<code>Mindguardians android doc.docx</code>	Portada, arquitectura, stack, dependencias, estructura de Firestore, flujos funcionales (autenticación, combate, Oráculo), instrucciones de instalación, seguridad y tabla de pantallas
<code>Mindguardians android readme.md</code>	Versión para GitHub con badges, instrucciones de setup y estructura de archivos

Web

Archivo	Descripción
<code>Mindguardians web doc.docx</code>	Mismas secciones adaptadas al proyecto Next.js, incluyendo variables de entorno y sincronización con Android
<code>Mindguardians web readme.md</code>	README listo para poner en el repositorio de GitHub

ARQ**Arquitectura y estructura de datos**

7. Arquitectura General

Flujo de autenticación

- Usuario escribe email/password en Login.
- Firebase Auth valida credenciales y retorna UID.
- Se carga el documento users/{uid} de Firestore.
- El estado local (GameState / React state) se popula con los datos reales.

Flujo de combate

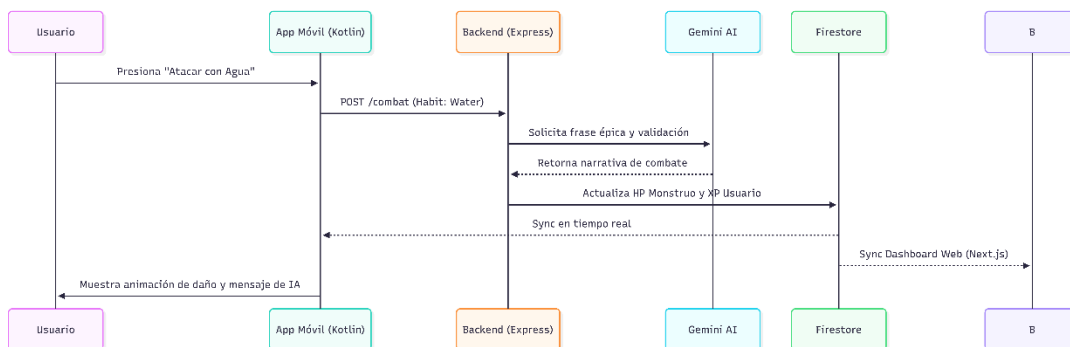
- El jugador selecciona un tipo de hábito y presiona atacar.
- GameViewModel actualiza HP del monstruo y aplica XP/oro localmente (respuesta inmediata).
- En segundo plano, FirebaseRepository.saveBattle() y saveUserData() persisten los cambios.
- Si el jugador sube de nivel, se recalcula totalXp para el ranking global.

Flujo del Oráculo

- El usuario escribe su hazaña de salud en el campo de texto.
- Se llama a GeminiRepository.consultOracle() con el prompt temático.
- Gemini responde con narración épica en 3 oraciones + bonificador de daño.
- La respuesta se muestra en la burbuja del chat y se actualiza el estado del juego.

Sincronización Web ↔ Android

Ambas plataformas comparten exactamente los mismos nombres de campos en Firestore, asegurando que los datos escritos desde Android sean legibles por la web y viceversa sin transformación adicional.



DEBUG**Challenges & Solutions**

8. Problemas encontrados y estrategias aplicadas

Problemas

- Al intentar preparar firebase y firestore, había errores al ejecutar ciertos comandos debido a que los permisos para ejecutarlos no estaban configurados correctamente.
- Utilizábamos una versión despreciada de Gemini por lo cual no generaba respuestas.
- Desde la aplicación Android, las consultas a Gemini terminaban con un error de timeout debido a que el tiempo de espera para las respuestas era muy corto.
- En ciertos puntos en la parte de Android, se producían errores al momento de ejecutar ciertas funciones específica del programa, pero no se indicaban con claridad, por lo que era difícil diagnosticar los problemas para ver que había fallado y causado el error.

Soluciones

- Se modificaron los permisos desde Firebase Console para permitir el uso de los comandos necesarios.
- Por medio de comandos encontramos todas las versiones de Gemini que funcionan y reemplazamos la versión despreciada por la que consideramos ser la mejor versión para nuestro proyecto.
- Se utilizó la librería okhttp para modificar el tiempo de espera concedido antes de concluir que es demasiado y señalar un error de timeout.
- Utilizamos Logcat de AndroidStudio para analizar donde fue que el error ocurrió y porque, permitiendo deducir si la causa fue en la parte de Android o si fue en la parte de firebase/firestore.

TODO**Pendientes y próximos pasos**

9. Pendientes y Próximos Pasos

Web

- Conectar GeminiOracle.tsx al endpoint real de Gemini (actualmente usa mensajes mock).
- Implementar registro de batallas desde la web.
- Conectar el ranking con datos reales de Firestore.
- Agregar Cloud Functions para lógica de servidor (validación de batallas, lógica de subida de nivel).

Android

- Pantalla de Login y Registro con Firebase Auth.
- Conectar datos del ranking al Firestore real.
- Manejo de errores de red con retry automático.
- Notificaciones push con Firebase Messaging (FCM).

General

- Configurar Cloud Functions (Express/Node.js) como backend intermedio.
 - Google OAuth como método de autenticación adicional.
 - Tests unitarios para ViewModels y repositorios.
 - CI/CD con GitHub Actions para builds automáticos.
-